

MANUAL TÉCNICO

OptiBottle

**Sistema de Información para la Optimización de la
Producción de Botellas Plásticas**

Proyecto:	Proyecto Integrador de Aprendizaje (PIA)
Materia:	Sistemas de Información
Equipo:	2
Tipo de empresa:	Mediana, fabricación y distribución de botellas plásticas
Lenguaje:	Python 3
Versión del documento:	1.0

Índice

1. Introducción
2. Objetivo del Sistema
3. Alcance y Justificación
4. Arquitectura del Sistema
5. Tecnologías Utilizadas
6. Modelo de Datos (Base de Datos)
7. Estructura del Código
8. Descripción de Módulos
9. Reglas de Negocio y Fórmulas
10. Instalación y Configuración
11. Pruebas Realizadas
12. Mantenimiento y Mejora Futura

1. Introducción

El presente manual técnico describe la arquitectura, componentes y funcionamiento interno del sistema **OptiBottle**, desarrollado como Proyecto Integrador de Aprendizaje (PIA) por el Equipo 2 para la unidad de aprendizaje de Sistemas de Información. El documento está dirigido a personal técnico encargado de la instalación, configuración, mantenimiento o evolución del sistema.

2. Objetivo del Sistema

Proporcionar una herramienta de software que permita a una empresa mediana dedicada a la fabricación y distribución de botellas plásticas **optimizar su área de producción**, mediante la captura sistemática de información de órdenes, inventarios y eventos de producción, así como la generación de indicadores que ayuden a identificar desperdicios, tiempos muertos y oportunidades de mejora.

3. Alcance y Justificación

Alcance:

El sistema cubre la gestión de órdenes de producción, control básico de inventario de materia prima, registro de producción por turno y la visualización de KPIs en un tablero (dashboard). No incluye módulos de ventas, facturación ni nómina, los cuales podrían integrarse como fases posteriores.

Justificación:

Una empresa mediana del giro plástico genera cientos de eventos productivos al día. Sin un sistema de información, es imposible identificar de manera objetiva dónde están las pérdidas. OptiBottle convierte datos dispersos en información útil para la toma de decisiones, permitiendo a la organización trabajar de manera más ordenada, eficiente y competitiva.

4. Arquitectura del Sistema

OptiBottle sigue una arquitectura **en tres capas**, característica de los sistemas de información modernos:

Capa	Componente	Responsabilidad
Presentación	main.py (Tkinter)	Interfaz gráfica de usuario, formularios y dashboard
Lógica de Negocio	Métodos de Database	Cálculo de KPIs, validaciones y reglas
Persistencia	SQLite (optibottle.db)	Almacenamiento físico de los datos

5. Tecnologías Utilizadas

Tecnología	Versión	Uso
Python	3.8+	Lenguaje de programación principal
Tkinter / ttk	Incluida en Python	Interfaz gráfica de usuario
SQLite	Incluida en Python	Base de datos local embebida
unittest	Incluida en Python	Pruebas unitarias automatizadas
ReportLab	(para PDFs)	Generación de los manuales en PDF

6. Modelo de Datos (Base de Datos)

El sistema utiliza tres tablas principales en SQLite:

Tabla: **materia_prima**

Campo	Tipo	Descripción
id	INTEGER PK	Identificador autoincremental
nombre	TEXT UNIQUE	Nombre del material
stock_actual	REAL	Cantidad disponible
stock_minimo	REAL	Umbral para alerta
unidad	TEXT	kg, ton, piezas, litros

Tabla: **ordenes_produccion**

Campo	Tipo	Descripción
id	INTEGER PK	Identificador autoincremental
codigo	TEXT UNIQUE	Código visible de la orden (ej. ORD-001)
producto	TEXT	Producto a fabricar
cantidad_planeada	INTEGER	Meta de producción
cantidad_producida	INTEGER	Acumulado real (calculado)
fecha_inicio	TEXT	Fecha planeada de inicio (YYYY-MM-DD)
fecha_fin	TEXT	Fecha de cierre
estado	TEXT	Planeada / En Proceso / Pausada / Finalizada
responsable	TEXT	Persona a cargo

Tabla: **registros_produccion**

Campo	Tipo	Descripción
id	INTEGER PK	Identificador autoincremental
orden_id	INTEGER FK	Orden a la que pertenece el registro
fecha	TEXT	Fecha del turno
turno	TEXT	Matutino / Vespertino / Nocturno

unidades_buenas	INTEGER	Piezas conformes
unidades_defectuosas	INTEGER	Piezas no conformes
tiempo_muerto_min	INTEGER	Minutos sin producir
causa_paro	TEXT	Causa del paro (si hubo)

7. Estructura del Código

El proyecto está organizado de la siguiente manera:

```
optibottle/  
├── src/  
│   ├── main.py          # Interfaz gráfica (Tkinter)  
│   └── database.py      # Capa de datos y lógica de negocio  
├── tests/  
│   └── test_database.py # Pruebas unitarias  
├── data/  
│   ├── optibottle.db    # Base de datos SQLite  
│   ├── cargar_datos_ejemplo.py  
│   └── README.md
```

8. Descripción de Módulos

8.1 database.py

Contiene la clase **Database**, encargada de:

- Crear las tablas si no existen al iniciar.
- Operaciones CRUD sobre materia prima, órdenes y registros.
- Cálculo de KPIs (eficiencia, % desperdicio, tiempo muerto).
- Detección de materiales por debajo del stock mínimo.
- Agrupación de causas de paro para análisis.

8.2 main.py

Contiene la clase **OptiBottleApp**, que construye la interfaz gráfica con cuatro pestañas:

- **Dashboard:** Tarjetas con KPIs, tabla de causas de paro y alertas.
- **Órdenes de Producción:** Alta, listado, cambio de estado y exportación a CSV.
- **Materia Prima:** Alta, actualización de stock y alertas de mínimo.
- **Registro de Producción:** Captura diaria por turno y orden.

9. Reglas de Negocio y Fórmulas

9.1 Indicadores Calculados

KPI	Fórmula
% de Desperdicio	$(\text{unidades_defectuosas} / \text{total_unidades}) \times 100$
Eficiencia de Calidad	$(\text{unidades_buenas} / \text{total_unidades}) \times 100$
Tiempo Muerto Total	$\Sigma \text{tiempo_muerto_min}$ de todos los registros
Alerta de Stock	$\text{stock_actual} < \text{stock_minimo}$

9.2 Validaciones Implementadas

- No se permiten valores negativos en cantidades, defectuosos o tiempo muerto.
- Las fechas deben estar en formato YYYY-MM-DD.
- Los códigos de orden y nombres de material son únicos.
- Al eliminar una orden, se eliminan sus registros (integridad referencial).

10. Instalación y Configuración

Requisitos previos:

- Python 3.8 o superior instalado.
- No se requieren dependencias externas (todo viene incluido en Python).

Pasos:

1. Descomprimir el archivo del proyecto.
2. (Opcional) Cargar datos de prueba ejecutando:

```
python cargar_datos_ejemplo.py
```

3. Iniciar la aplicación:

```
python src/main.py
```

11. Pruebas Realizadas

Se desarrollaron pruebas unitarias usando el módulo **unittest** de Python, verificando los métodos críticos de la capa de datos. Se ejecutaron 11 pruebas cubriendo: alta de materiales, actualización de stock, detección de stock bajo, creación y modificación de órdenes, registro de producción, cálculo de métricas y agrupación de causas de paro.

#	Prueba	Resultado
1	test_actualizar_estado_orden	OK
2	test_actualizar_stock	OK
3	test_agregar_material	OK
4	test_calcular_metricas	OK
5	test_causas_paro_frecuentes	OK
6	test_crear_orden	OK
7	test_eliminar_material	OK
8	test_eliminar_orden_borra_registros	OK
9	test_materiales_bajo_minimo	OK
10	test_metricas_sin_datos	OK
11	test_registrar_produccion	OK

Resultado global: 11 pruebas ejecutadas, 11 exitosas, 0 fallas. Tiempo total: 0.22 segundos.

12. Mantenimiento y Mejora Futura

Posibles ampliaciones recomendadas:

- Migración a una arquitectura cliente-servidor con PostgreSQL o MySQL para uso multi-usuario.
- Integración con sensores IoT en la línea de producción para captura automática.
- Módulo de pronóstico de demanda con machine learning.
- Generación automática de reportes en PDF y envío por correo.
- Roles y permisos de usuario (operador, supervisor, gerente).
- Versión web responsive con Flask o Django.